University of Tehran
College of Interdisciplinary Science and Technologies

Soft Computing

2nd Assignment

Name: Mahan Madani
Student ID: 830504035

Contents

1 Loss Functions **3**
- 1.1 Mathematical Definitions . . . 3
- 1.2 Advantages and Disadvantages . . . 3
- 1.3 Comparison Table . . . 5
- 1.4 Use Cases . . . 5

2 Image Compression with MLP Autoencoders **6**
- 2.1 Hidden Layer Neurons . . . 7
  - 2.1.a 4 Neurons . . . 8
  - 2.1.b 16 Neurons . . . 8
  - 2.1.c 32 Neurons . . . 8
  - 2.1.d Hidden Layer Size Comparison . . . 9
- 2.2 Block Size . . . 9
  - 2.2.a $4 * 4$ Blocks . . . 10
  - 2.2.b $8 * 8$ Blocks . . . 10
  - 2.2.c $16 * 16$ Blocks . . . 10
  - 2.2.d Block Size Comparison . . . 11
- 2.3 Backpropagation with Momentum . . . 11

3 Feature Extractions and Classification with MLP **12**
- 3.1 Train Autoencoder . . . 13
  - 3.1.a Hidden Layer Neurons . . . 13
  - 3.1.b Model Architecture . . . 14
  - 3.1.c ReLU Activation Function . . . 15
  - 3.1.d Learning Rate . . . 16
  - 3.1.e Batch Gradient Descent . . . 17
  - 3.1.f Test Images with the Largest and Smallest Loss . . . 19
- 3.2 Train Classifier . . . 20
  - 3.2.a Classifiers with different Hidden Layer Size . . . 21
  - 3.2.b Comparison with Official Results . . . 22

References **23**

List of Figures

1 Sample image divided into 64 blocks of size 32. 7
2 Sample image reconstruction — Hidden layer size = 4. 8
3 Sample image reconstruction — Hidden layer size = 16. 8
4 Sample image reconstruction — Hidden layer size = 32. 8
5 Comparison table of models with different hidden layer sizes 9
6 Comparison plot of models with different hidden layer sizes 9
7 Sample image reconstruction — Block size = 4. 10
8 Sample image reconstruction — Block size = 8. 10
9 Sample image reconstruction — Block size = 16. 10
10 Comparison table of models trained with different block sizes 11
11 Comparison plot of models trained with different block sizes 11
12 Comparison table of models trained with different momentum values . . 12
13 Comparison plot of models trained with different momentum values . . 12
14 Autoencoder model with hidden layer size 64 13
15 Autoencoder model with hidden layer size 128 13
16 Autoencoder model 768 - 256 - 64 - 256 - 768 14
17 Autoencoder model 768 -256 - 128 - 64 - 128 - 256 - 768 14
18 Comparison table of the first 4 models 15
19 Train MSE over epochs on Validation data 15
20 Autoencoder model with ReLU Activation function 15
21 Comparison of Sigmoid vs ReLU 16
22 Autoencoder model with Learning Rate = 0.01 16
23 Autoencoder model with Learning Rate = 0.000001 17
24 Comparison of learning rate 17
25 Autoencoder model trained with batch size = 8 18
26 Autoencoder model trained with batch size = 16 18
27 Comparison of learning rate 19
28 Top 5 images with the smallest loss 19
29 The 5 images with the largest loss 20
30 Classifier model with hidden layer size = 64 21
31 Classifier model with hidden layer size = 32 22

List of Tables

1 Loss function comparison 5

Question 1. Loss Functions

1.1 Mathematical Definitions

MAE (Mean Absolute Error)

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

MSE (Mean Squared Error)

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

RMSE (Root Mean Squared Error)

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

R² (Coefficient of Determination)

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

Cross-Entropy Loss

$$L = -\left[y\log(\hat{y}) + (1-y)\log(1-\hat{y})\right]$$

1.2 Advantages and Disadvantages

In this section, we will compare the aforementioned function in terms of sensitivity to outlier datapoints, differentiability, and whether they are more suited for classification problems or regression problems.

MAE
- Advantages

- Robust to outliers (linear penalty).
- Easy to interpret.

- Disadvantages

- Not differentiable at zero.
- Small and large errors are treated equally.

Outlier sensitivity: Low
Smoothness: Non-smooth at zero
Usage: Regression

MSE
- Advantages

- Strong penalty for large errors.
- Smooth and differentiable, ideal for gradient descent.

- Disadvantages

- Highly sensitive to outliers.
- Less interpretable than MAE.

Outlier sensitivity: Very high
Smoothness: Smooth
Usage: Regression

RMSE
- Advantages

- Same units as the target variable (better interpretability).
- Behaves similarly to MSE but easier to interpret.

- Disadvantages

- Still highly sensitive to outliers.
- Square-root introduces an additional nonlinearity.

Outlier sensitivity: High
Smoothness: Smooth
Usage: Regression

R^2
- Advantages

- Measures the percentage of variance the model explains.
- Scale-free and easy to compare across datasets.

- Disadvantages

- Misleading for nonlinear models.
- Undefined for some cases like constant models.

Outlier sensitivity: High
Smoothness: Smooth (differentiable when denominator $\sum(y_i - \bar{y})^2 \neq 0$)
Usage: Regression evaluation only Not a loss function—only an evaluation metric.

Cross-Entropy Loss
- Advantages

- Ideal for classification tasks.
- Strong gradient when predictions are wrong (fast convergence).
- Works naturally with softmax/sigmoid.

- Disadvantages

- Sensitive to label noise.
- Not intuitively interpretable.

Outlier sensitivity: High
Smoothness: Smooth (differentiable for $\hat{y} \in (0, 1)$; numerical implementations clip values to avoid $\log 0$)
Usage: Classification (binary or multi-class)

1.3 Comparison Table

Metric	Outlier Sensitivity	Smooth?	Reg	Class	Notes
MAE	Low	No	Yes	No	Robust errors
MSE	Very High	Yes	Yes	No	Standard regression loss
RMSE	High	Yes	Yes	No	Same units as target
R^2	High	Yes (if $y \neq 0$)	Yes	No	Explained variance
Cross-Entropy	High	Yes ($\hat{y} \in (0, 1)$)	No	Yes	Best for classification

Table 1: Loss function comparison

1.4 Use Cases

MAE

- Predicting house prices in the presence of outliers.
- Any regression problem requiring robustness.

MSE / RMSE

- Weather prediction models.
- Energy consumption forecasting.
- Image denoising or reconstruction (penalizing large pixel errors).

R^2

- Evaluating regression models in economics.
- Scientific modeling where variance explanation matters.

Cross-Entropy Loss

- Binary classification (spam filter, fraud detection).
- Multi-class classification (e.g., Fashion-MNIST).
- NLP text classification.

Question 2. Image Compression with MLP Autoencoders

The goal of the question is to manually implement an autoencoder neural network using the NumPy Library, and compare the effects of hyperparameters such as hidden-layer size, model input size, and optimization variants (momentum vs non-momentum).

File Structure

All experiments and models required by this question are implemented in these two main files:

- **Q2.ipynb**: the full experimental pipeline, data preprocessing, model training, evaluation, and plotting.
- **MLP.py**: a complete custom implementation of multilayer perceptrons (MLPs), activation functions, training logic, and utilities. All experiments rely only on this custom implementation.

The MLP.py File

This file defines all the neural network functions and classes:

- `MLP` class:
  - configurable layer sizes
  - forward propagation
  - backpropagation
  - gradient descent (with batch updates) and optional momentum
  - multiple activation functions and loss functions
- **Layer classes**: fully connected linear layers
- **Activation functions**: ReLU, Leaky ReLU, Linear, Sigmoid, Tanh, Softmax
- **Loss functions**: MSE for reconstruction, cross-entropy for classification
- **Utility functions**: weight initialization, PSNR calculation, etc.

The architecture is deliberately modular, making it easy to swap activation functions, insert intermediate layers, or adjust dimensions.

Dividing Images into $n * n$ blocks

The dataset provided for this task consists of grayscale images of size $256 * 256$, with 91 being used for the training process and 5 used for model evaluation.

In order to obtain better results for the autoencoder model, each image is divided into $n * n$ blocks, which are then converted to vectors of size n^2. The training data is dynamically generated based on the block size for each experiment. Figure 1 showcases one image being divided into 64 blocks of size 32.

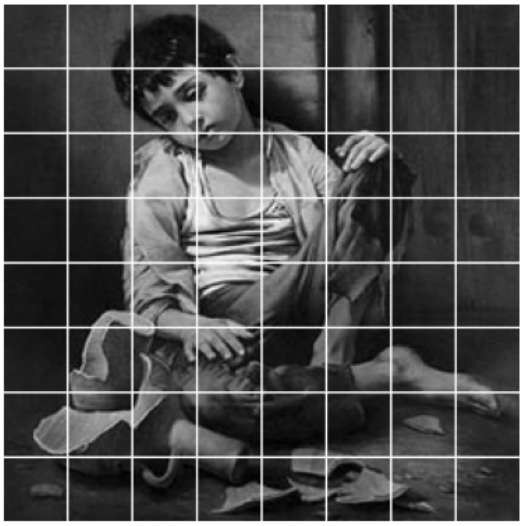


Figure 1: Sample image divided into 64 blocks of size 32.

Train and Test loss

In my experiments in the provided code, the training loss reported at each epoch was calculated during the model's training process. Meaning that the weight updates applied after each batch only affect the next batches. The test loss however, was calculated at the end of the epoch where all the weight updates had already been performed. This resulted in the test loss of the epoch to be smaller compared to the training loss, because the model was tested with updated weights.

Additionally, the number of sample images used for testing the model is significantly smaller than the number of training samples, and it includes simpler images as well, resulting in **better loss and PSNR results for the testing set in general.**

2.1 Hidden Layer Neurons

In this section, three models are created with different hidden layer sizes (4, 16, 32). The general structure of each model is:

```
    Multi-Layer Perceptron Details:
Layer 1:  Linear Block (64, hidden_layer_size), Activation:  Sigmoid
Layer 2:  Linear Block (hidden_layer_size, 64), Activation:  Linear
Learning Rate:  0.001
Momentum:  0.0
Loss Function:  MSE
Weight Initialization Type:  xavier
```

The models were trained using batch gradient descent (batch size = 16) over 50 epochs and the input images were divided into $8*8$ blocks, which were then vectorized.

2.1.a 4 Neurons

Figure 2: Sample image reconstruction — Hidden layer size = 4.

2.1.b 16 Neurons

Figure 3: Sample image reconstruction — Hidden layer size = 16.

2.1.c 32 Neurons

Figure 4: Sample image reconstruction — Hidden layer size = 32.

2.1.d Hidden Layer Size Comparison

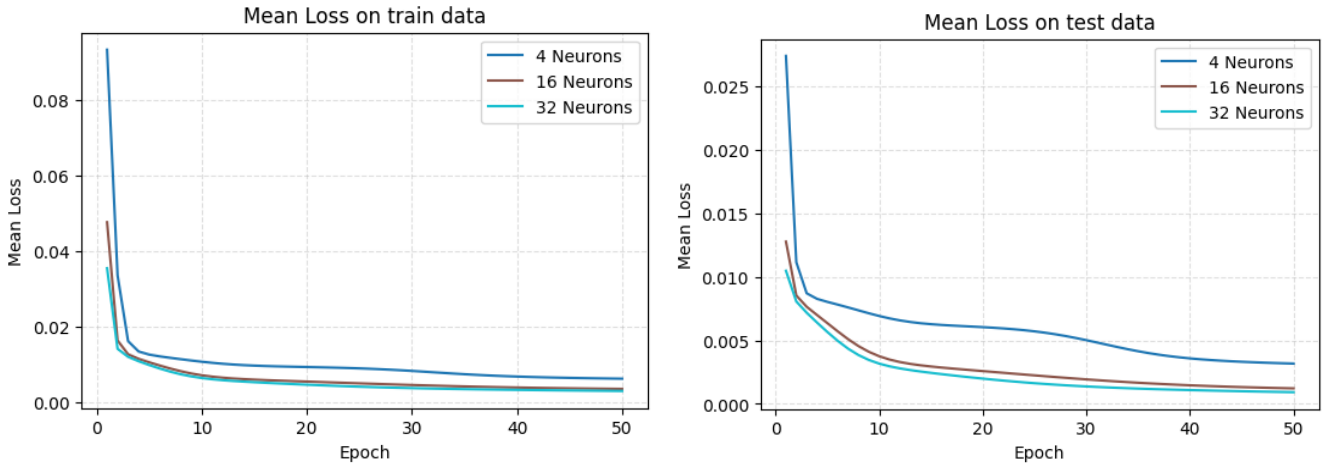
Figure 5 displays a table comparing the 3 models using various metrics. The model with **32 neurons** in its hidden layers had the best performance overall, with the smallest loss values and the largest PSNR (Peak Signal-to-Noise Ratio). A higher PSNR value indicates a better quality reconstruction because the difference (noise) between the original and reconstructed image is smaller. The value is expressed in decibels.

Figure 6 showcases the loss history of the models over their training process. The model with 32 neurons had the most stable training and converged the fastest.

```
Model Evaluation Summary:

        Model        | Train Loss | Test Loss  | Train PSNR  | Test PSNR
-------------------------------------------------------------------------
  Hidden Layers = 4   |  0.006343  |  0.003172  |  23.979466  | 25.886967
  Hidden Layers = 16  |  0.003638  |  0.001219  |  27.502666  | 30.412798
  Hidden Layers = 32  |  0.003085  |  0.000917  |  28.482511  | 31.664439
```

Figure 5: Comparison table of models with different hidden layer sizes



(**a**) Train MSE over epochs (**b**) Test MSE over epochs

Figure 6: Comparison plot of models with different hidden layer sizes

2.2 Block Size

In this section, three models are created with different input image block sizes ($4*4, 8*8, 16*16$). The general structure of each model is:

```
    Multi-Layer Perceptron Details:
Layer 1:  Linear Block (n*n, 32, Activation:  Sigmoid
Layer 2:  Linear Block (32, n*n), Activation:  Linear
Learning Rate:  0.001
Momentum:  0.0
Loss Function:  MSE
Weight Initialization Type:  xavier
```

The models were trained using batch gradient descent (batch size = 16) over 50 epochs and the hidden layer size was set to 32, as it was indicated to be the best by the previous experiment.

2.2.a $4 * 4$ Blocks

Figure 7: Sample image reconstruction — Block size = 4.

2.2.b $8 * 8$ Blocks

Figure 8: Sample image reconstruction — Block size = 8.

2.2.c $16 * 16$ Blocks

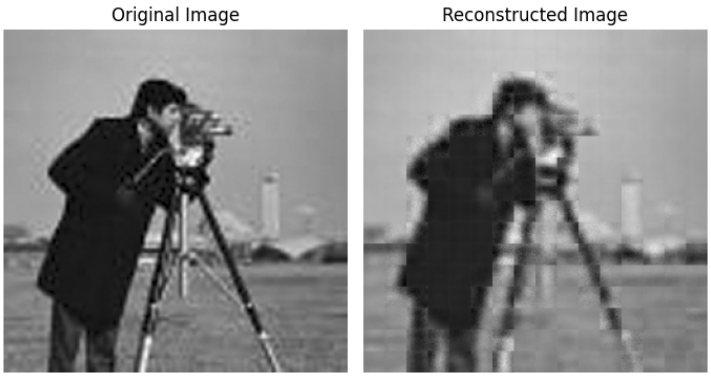


Figure 9: Sample image reconstruction — Block size = 16.

2.2.d Block Size Comparison

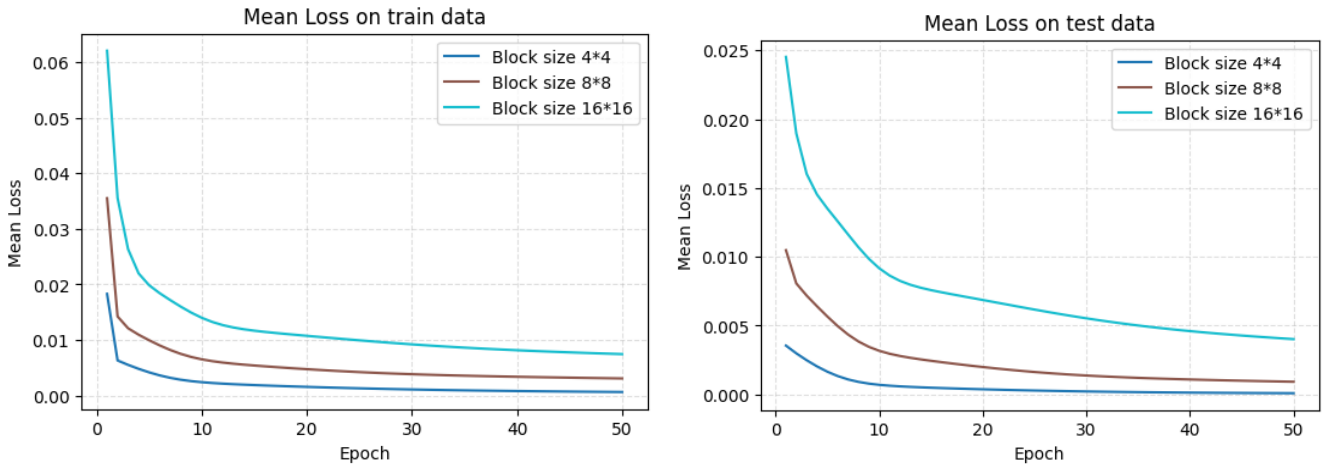
Figure 10 displays a table comparing the 3 models using various metrics. The model trained using $4 * 4$ blocks performed the best, however this model's architecture is $16 - 32 - 16$ which means the latent space of the model is actually larger than the input images. While this model shouldn't be used for image compression (because it would instead increase the dimensions of the image) it can be used for image denoising tasks.

Figure 6 showcases the loss history of the models over their training process. The model using $4 * 4$ blocks converged the fastest, and the model using $16 * 16$ blocks performed much worse than the other two, producing noticeably worse reconstructions (as shown in Figure 9).

```
Model Evaluation Summary:

        Model        | Train Loss | Test Loss  | Train PSNR  | Test PSNR
-------------------------------------------------------------------------
  Block Size = 4     |  0.000642  |  0.000086  |  37.071327  | 41.736679
  Block Size = 8     |  0.003085  |  0.000917  |  28.482511  | 31.664439
  Block Size = 16    |  0.007461  |  0.004017  |  23.305007  | 24.995006
```

Figure 10: Comparison table of models trained with different block sizes



(**a**) Train MSE over epochs

(**b**) Test MSE over epochs

Figure 11: Comparison plot of models trained with different block sizes

2.3 Backpropagation with Momentum

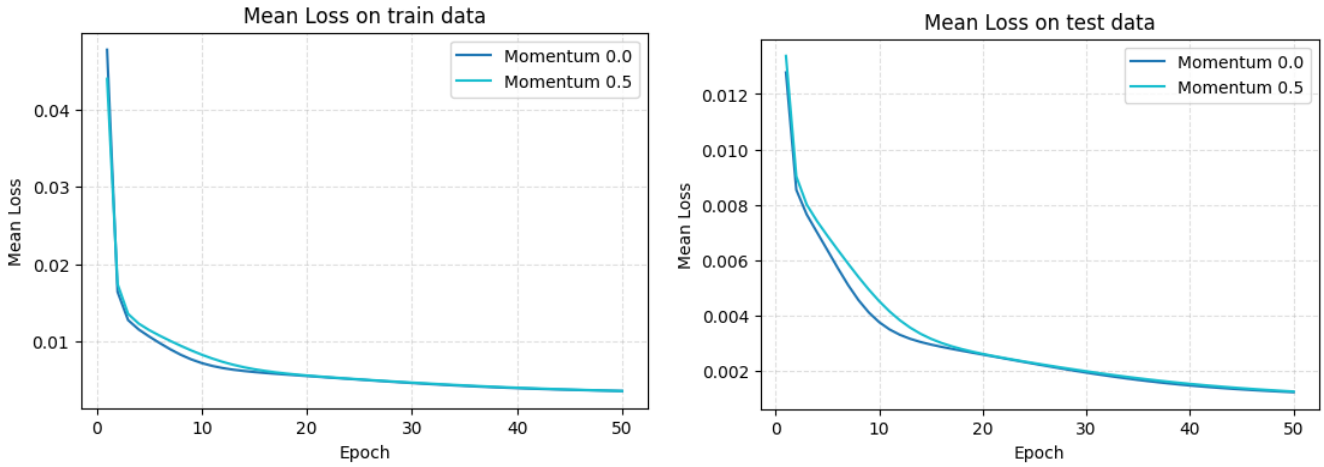
Momentum reduces oscillations and improves the stability of the training process. It can lead to faster convergence, especially early in training, and sometimes achieves a slightly better final PSNR.

Comparing two models with and without using momentum (Figure 12), we can see that slightly better results can be achieved with $momentum_coefficient = 0.5$ with a smoother loss curve (Figure 13), but it has also resulted in a longer convergence time for the model.

```
Model Evaluation Summary:

        Model        | Train Loss | Test Loss  | Train PSNR  |  Test PSNR  |  Convergence Time
-------------------------------------------------------------------------------------------------
    Momentum = 0.0    |  0.003638  |  0.001219  |  27.502666  |  30.412798  |     10 Epochs
    Momentum = 0.5    |  0.003644  |  0.001244  |  27.382973  |  30.267065  |     15 Epochs
```

Figure 12: Comparison table of models trained with different momentum values



(**a**) Train MSE over epochs (**b**) Test MSE over epochs

Figure 13: Comparison plot of models trained with different momentum values

Question 3. Feature Extractions and Classification with MLP

The goal of the question is to manually implement an autoencoder neural network trained on the Fashion MNIST dataset to reconstruct the images in the dataset. Then, utilize the autoencoder's latent space as input features to a classification model used for labeling the data.

File Structure

Similar to question 2, all experiments and models required by this question are implemented in these two main files:

- **Q3.ipynb**
- **MLP.py**

Ultimately, the autoencoder will act as a **feature extractor** for the classification MLP. The feature extractor is often implemented using more advanced architectures, such as CNNs, for more robust feature extraction from image data.

Dataset

The Fashion MNIST dataset was loaded and preprocessed using custom functions for reading IDX format files. The dataset contains 60,000 training images and 10,000 test images of 10 fashion categories. All images are of size $28 * 28$, resulting in vectors of size 768. Labels were converted to one-hot encoded vectors for use alongside the softmax activation function.

3.1 Train Autoencoder

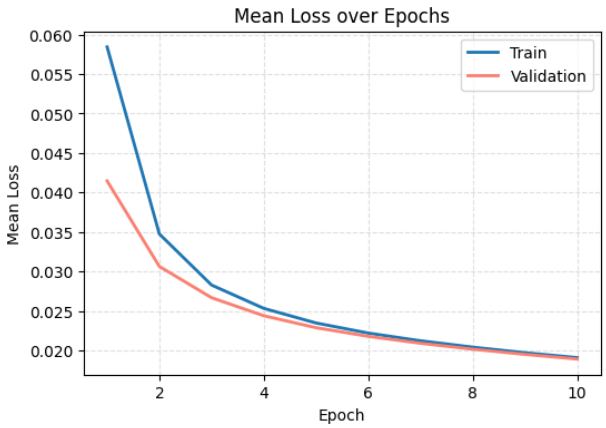
In this section, the training set was split into training (80%) and validation (20%) sets using a custom function that shuffles data with a fixed seed for reproducibility. The general structure of each created model is:

```
    Multi-Layer Perceptron Details:
Layer 1:  Linear Block (768, hidden_layer_sizes), Activation:  Sigmoid or ReLU
Layer 2:  Linear Block (hidden_layer_sizes, 768), Activation:  Sigmoid or ReLU
Learning Rate:  0.001 | 0.01 | 0.000001
Momentum:  0.0
Loss Function:  MSE
Weight Initialization Type:  xavier
```

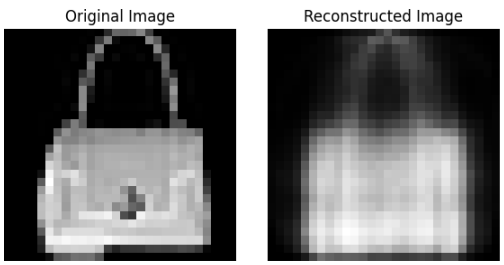
The models are all trained using SGD (batch size = 1) until the section where batch size is changed. The models are tested on the **validation dataset**, without ever seeing its data.

3.1.a Hidden Layer Neurons

For this section, 2 models were created and compared. Each model has 1 hidden layer (64 vs 128 neurons).
Figure 14 shows the results of the model with 64 neurons, and Figure 15 shows the results of the model with 128 neurons. The second model performs slightly better.

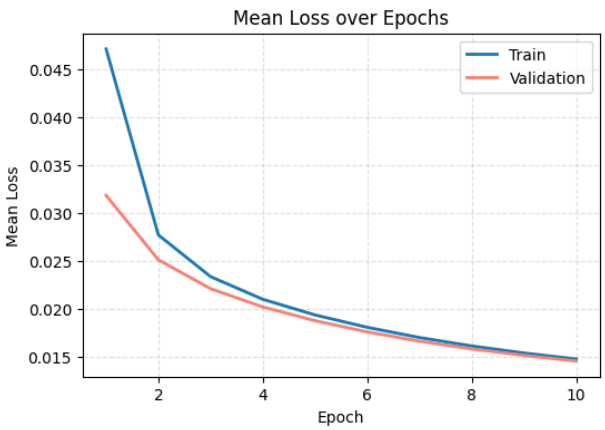


(**a**) Train and test MSE over epochs

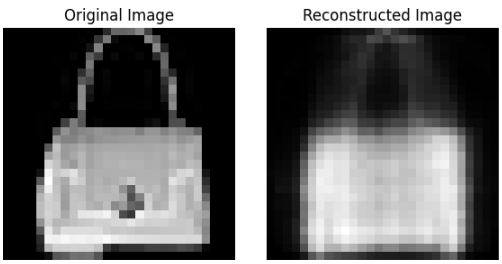


(**b**) Sample image - Mean PSNR = 17.764995

Figure 14: Autoencoder model with hidden layer size 64



(**a**) Train and test MSE over epochs



(**b**) Sample image - Mean PSNR: 18.976234

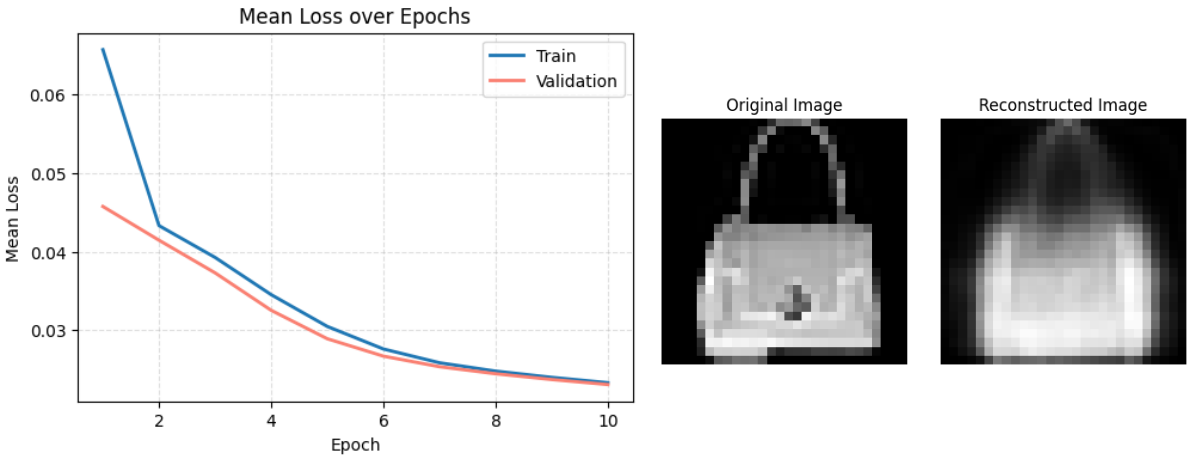
Figure 15: Autoencoder model with hidden layer size 128

The relationship between hidden layer size and reconstruction quality follows expected patterns:

- Smaller bottlenecks (64 neurons) force more aggressive compression, losing fine details
- Larger bottlenecks (128 neurons) preserve more information, enabling better reconstruction
- The 4:1 compression ratio (784→128) provides a good balance between compression and quality

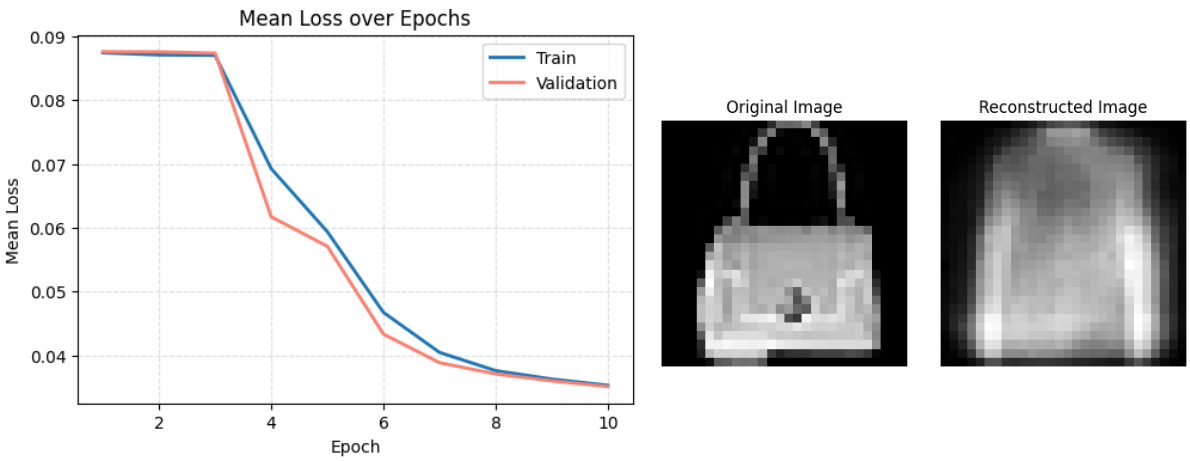
3.1.b Model Architecture

For this section, 2 additional models were created and compared. The first model (Figure 16) has 3 hidden layers and the second model (Figure 17) has 5 hidden layers. Both of the models have slower convergence times and worse performance compared to the previous models wtih 1 hidden layer.



(**a**) Train and test MSE over epochs (**b**) Sample image - Mean PSNR: 16.872900

Figure 16: Autoencoder model 768 - 256 - 64 - 256 - 768



(**a**) Train and test MSE over epochs (**b**) Sample image - Mean PSNR: 15.185058

Figure 17: Autoencoder model 768 -256 - 128 - 64 - 128 - 256 - 768

We can compare the 4 models created so far to choose the best architecture. Figure 18 and Figure 19 compare multiple metrics for each. The model with 1 hidden layer (size 128) seems to achieve the best results. The other model may perform better if trained for longer epochs, but with just 10 epochs of training, the **3-layer autoencoder 768 - 128 - 768** is the chosen model.

```
Model Evaluation Summary:

        Model                    | Train Loss |  Val Loss  | Val PSNR
-------------------------------------------------------------------------
 H: 64                           |  0.019086  |  0.018910  | 17.764995
 H: 128                          |  0.014803  |  0.014601  | 18.976234
 H: 256 - 64 - 256               |  0.023300  |  0.023067  | 16.872900
 H: 256 - 128 - 64 - 128 - 256   |  0.035293  |  0.035103  | 15.185058
```

Figure 18: Comparison table of the first 4 models

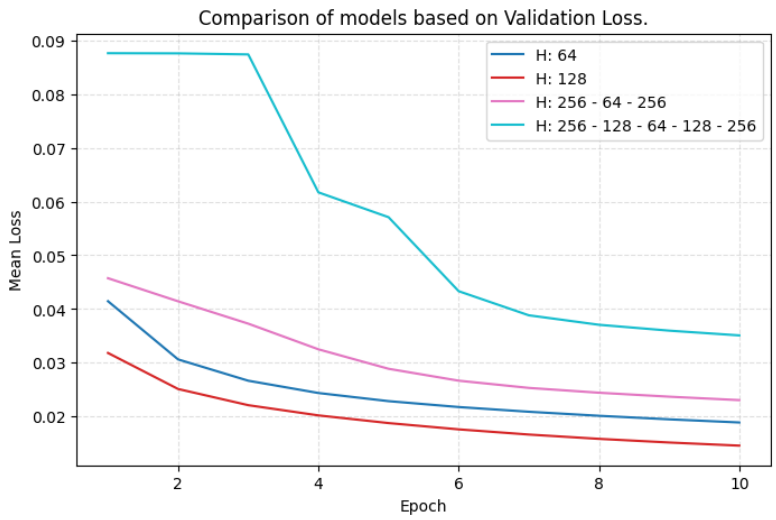
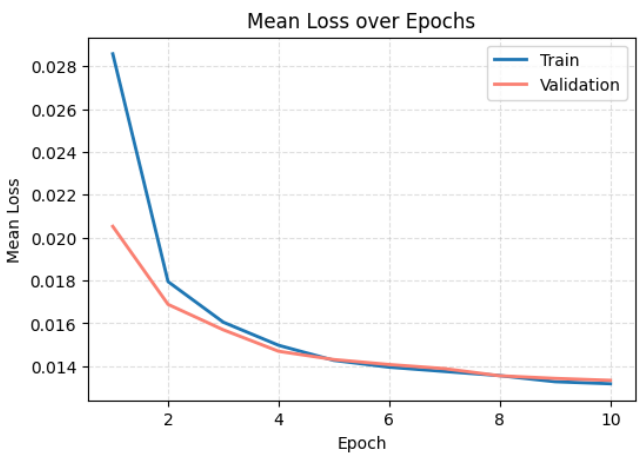


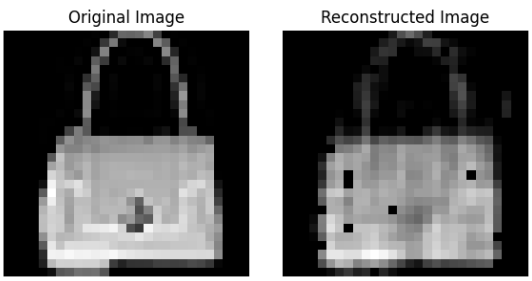
Figure 19: Train MSE over epochs on Validation data

3.1.c ReLU Activation Function

Figure 20 showcases the results of replacing the activation functions of the model with ReLU.



(**a**) Train and test MSE over epochs



(**b**) Sample image - Mean PSNR: 19.506506

Figure 20: Autoencoder model with ReLU Activation function

Replacing the Sigmoid function in the previous model with ReLU activation has multiple benefits, including simpler derivatives, reduced vanishing gradient issues, and faster convergence. The last benefit is clearly shown in Figure 21.

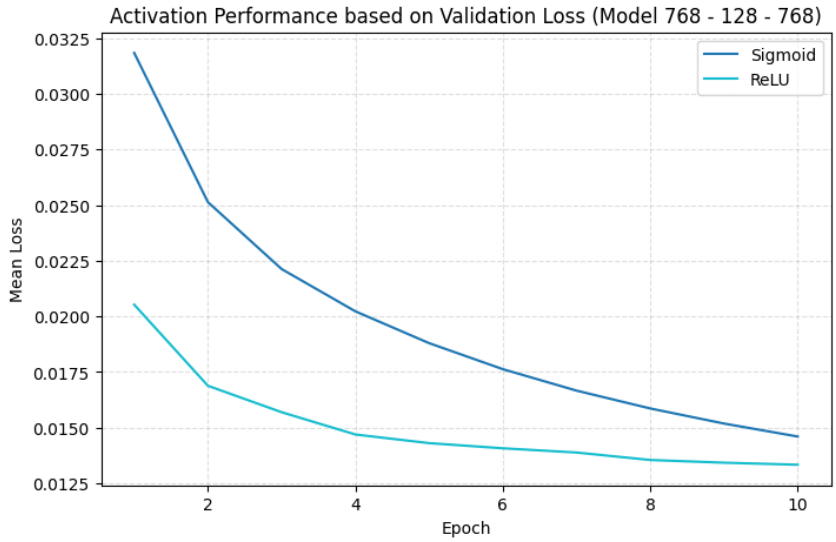
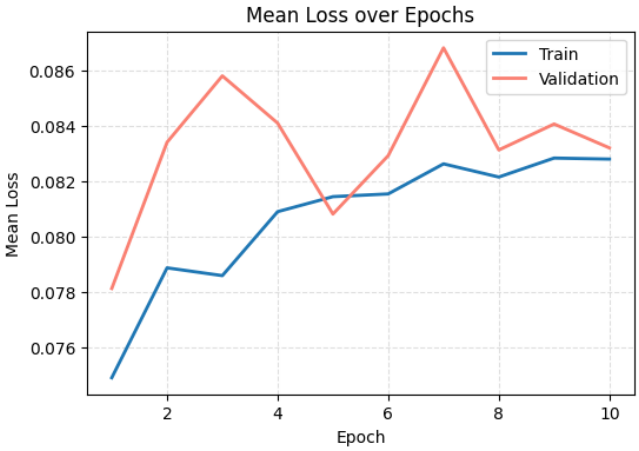


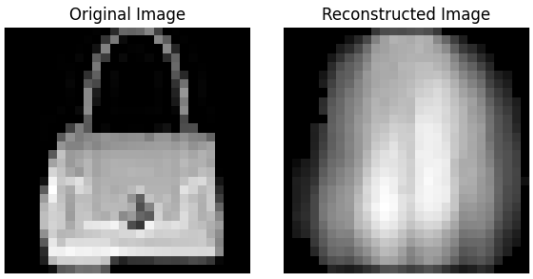
Figure 21: Comparison of Sigmoid vs ReLU

3.1.d Learning Rate

Changing the learning rate has a direct impact on the stability and speed of training. Figure 22 displays the training process of the model with learning rate set to 0.01, and Figure 23 is the same model with learning rate set to 0.000001.

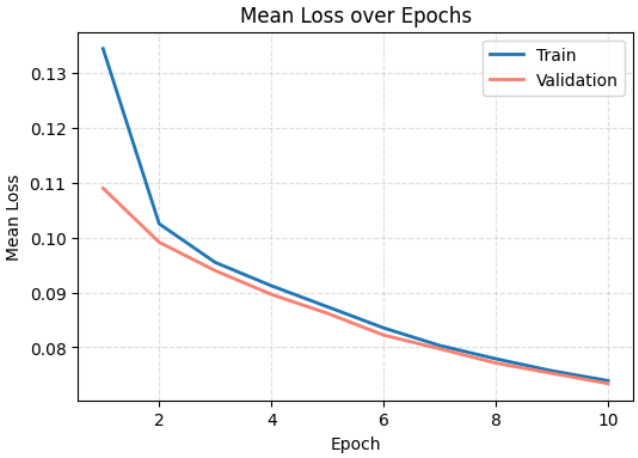


(**a**) Train and test MSE over epochs

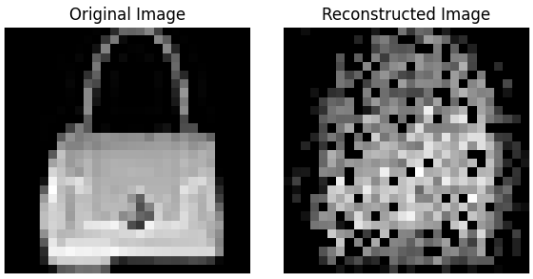


(**b**) Sample image - Mean PSNR: 11.136798

Figure 22: Autoencoder model with Learning Rate = 0.01



(a) Train and test MSE over epochs



(b) Sample image - Mean PSNR: 11.873826

Figure 23: Autoencoder model with Learning Rate = 0.000001

Figure 24 compares the model's performance over different learning rate.

Increasing the learning rate to **0.01** caused the optimization process to become unstable, with the loss oscillating or even rising instead of decreasing. On the other hand, reducing the learning rate to **0.000001** made the model converge extremely slowly, requiring far more epochs to achieve acceptable performance.

In practice, a learning rate of **0.001** provided the best balance, allowing the model to learn efficiently while maintaining stable and consistent decreases in the loss.

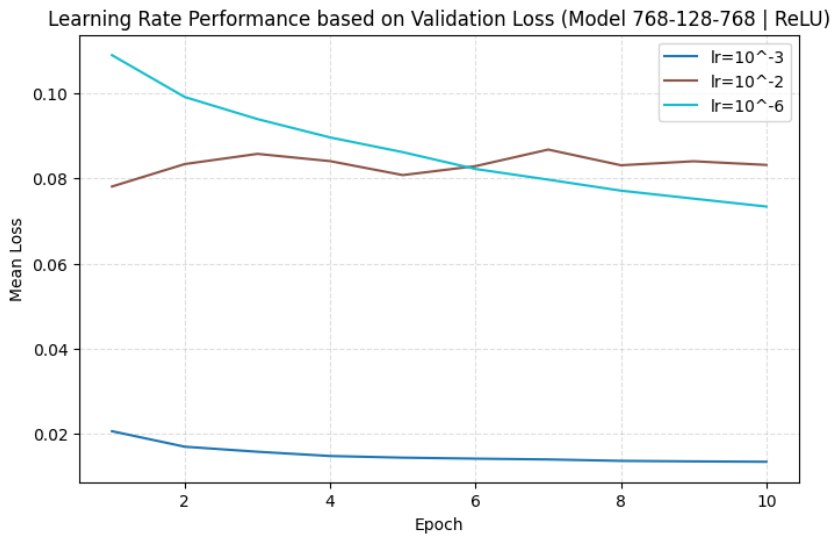
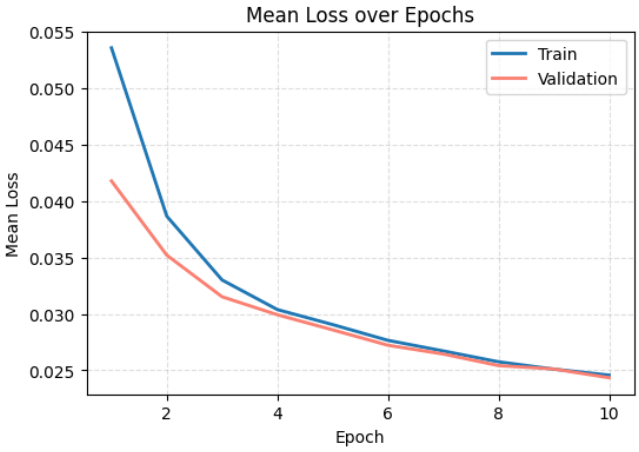


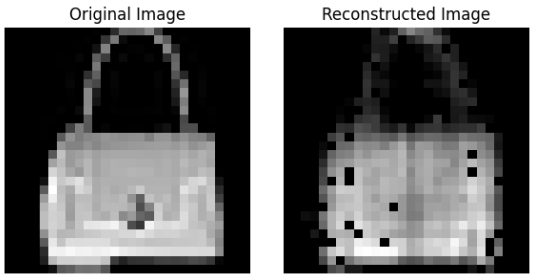
Figure 24: Comparison of learning rate

3.1.e Batch Gradient Descent

Batch gradient descent computes the gradient using a small subset of the training data instead of the entire dataset. Figure 25 displays the training process of the model with batch size set to 8, and Figure 23 is the same model with batch size set to 16.

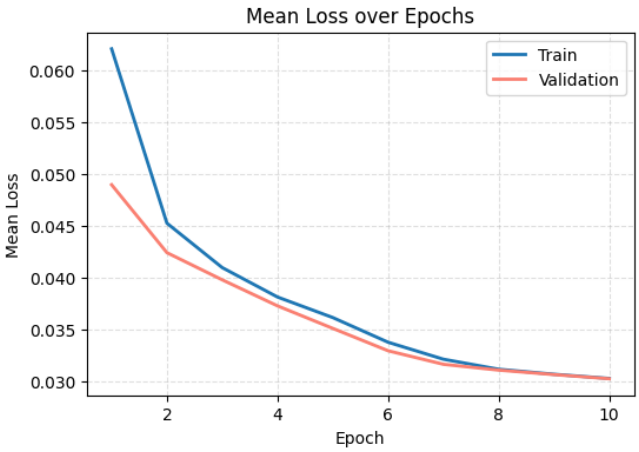


(**a**) Train and test MSE over epochs

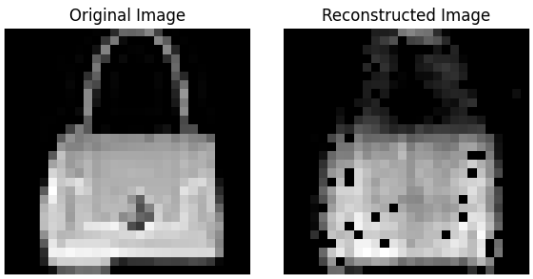


(**b**) Sample image - Mean PSNR: 16.875466

Figure 25: Autoencoder model trained with batch size = 8



(**a**) Train and test MSE over epochs



(**b**) Sample image - Mean PSNR: 15.855673

Figure 26: Autoencoder model trained with batch size = 16

Comparing the three approaches (Figure 27), we can see that using batch sizes like 8 or 16 provides a good balance between noisy single-sample updates and slow full-dataset updates. As a result, training becomes more stable than pure stochastic gradient descent, while convergence is slightly slower.

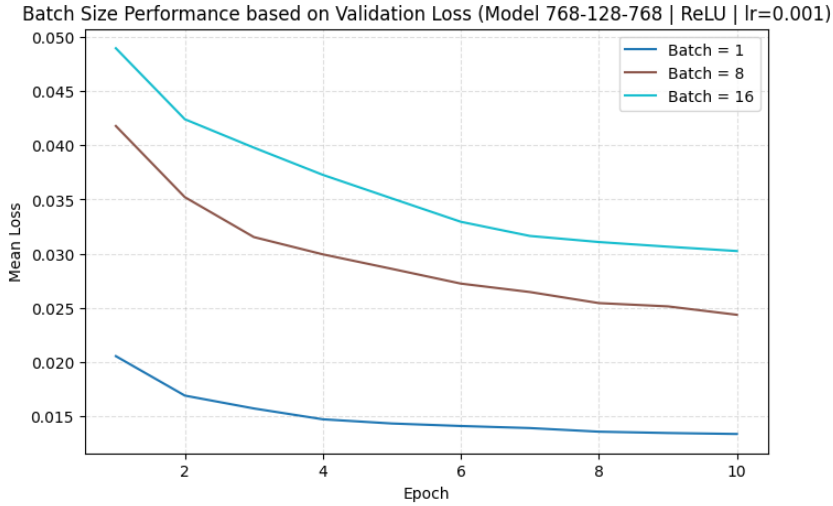
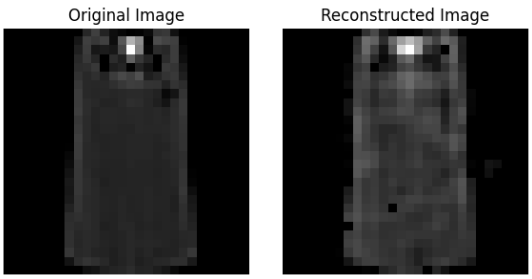


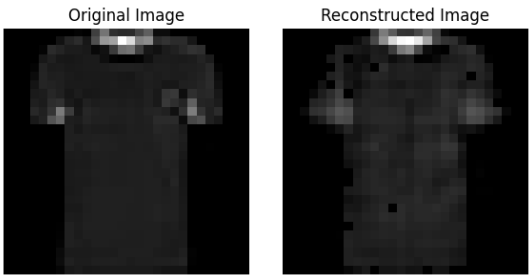
Figure 27: Comparison of learning rate

3.1.f Test Images with the Largest and Smallest Loss

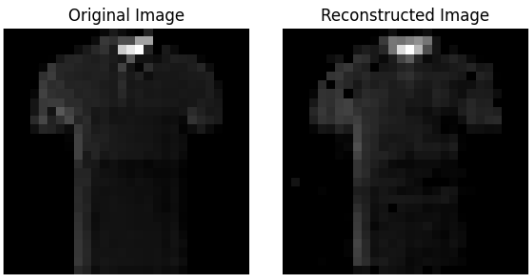
Figure 28 displays the top 5 images from the test dataset with the smallest loss when tested with the best model obtained from the previous sections.



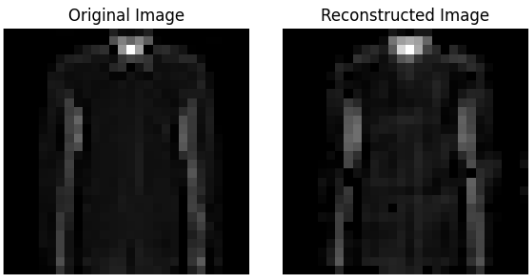
(**a**) Loss: 0.001294 - PSNR: 28.877916



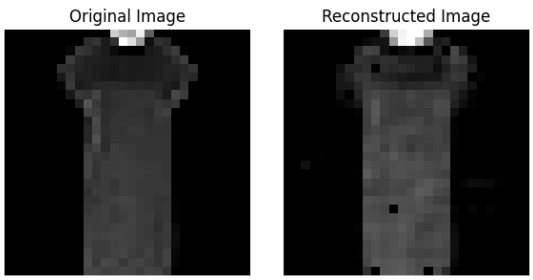
(**b**) Loss: 0.001324 - PSNR: 28.780905



(**c**) Loss: 0.001337 - PSNR: 28.736078



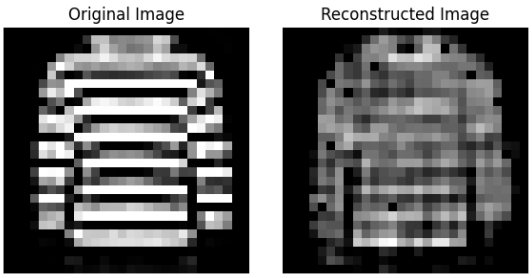
(**d**) Loss: 0.001401 - PSNR: 28.532790



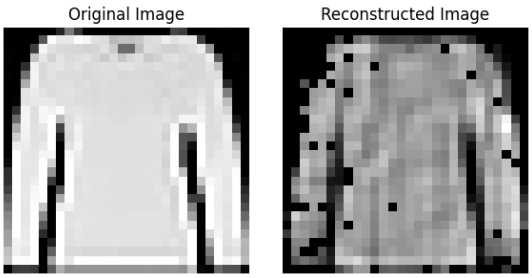
(**e**) Loss: 0.001466 - PSNR: 28.336866

Figure 28: Top 5 images with the smallest loss

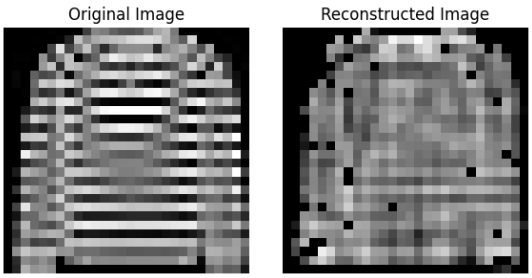
Figure 29 displays the 5 images from the test dataset with the largest loss when tested with the best model obtained from the previous sections.



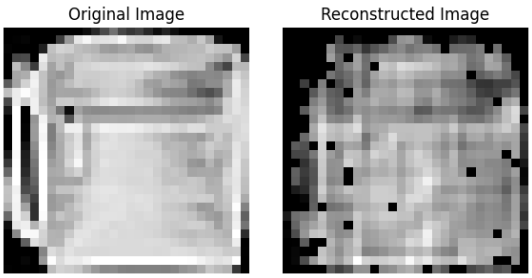
(**a**) Loss: 0.072571 - PSNR: 11.392330



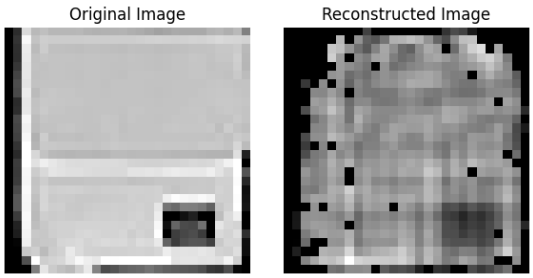
(**b**) Loss: 0.075181 - PSNR: 11.238877



(**c**) Loss: 0.088446 - PSNR: 10.533217



(**d**) Loss: 0.095487 - PSNR: 10.200527



(**e**) Loss: 0.001466 - PSNR: 28.336866

Figure 29: The 5 images with the largest loss

3.2 Train Classifier

Using the autoencoder from the last section, and a simple 3-layer MLP, we can create an image classification system for Fashion MNIST dataset.

The feature extractor is built from the best autoencoder model - the latent layer is the output of the model. Note that the feature extractor is not trained here, it is only used to extract features based on its previous parameters.

For this model, we use the entire train dataset to train the model and use the test dataset to evaluate the model.

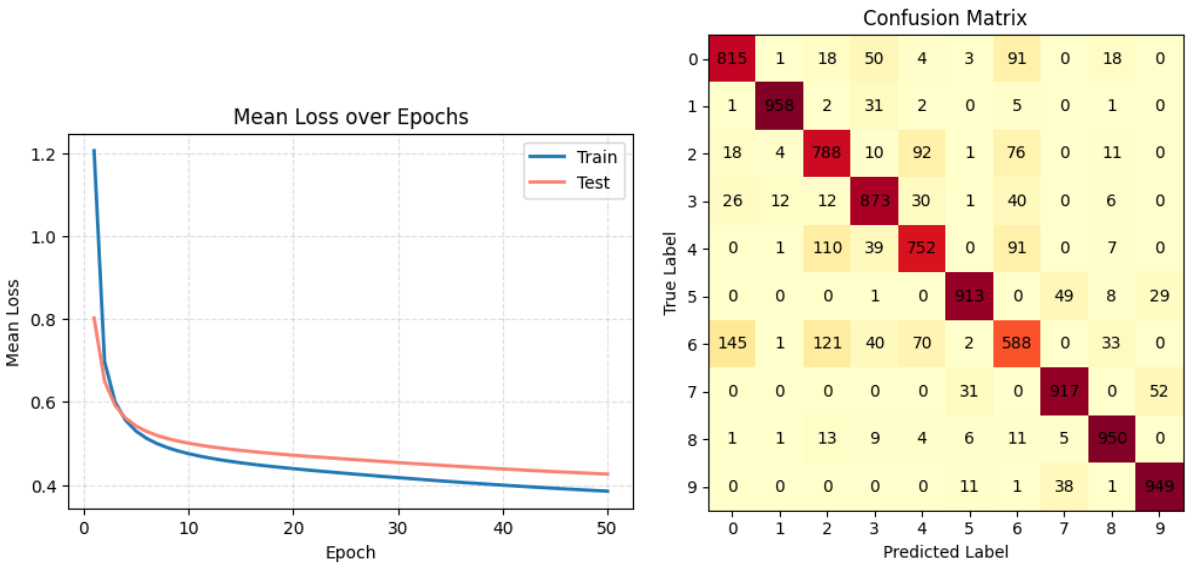
3.2.a Classifiers with different Hidden Layer Size

Cross-entropy loss and the Softmax activation function are used in the classifier as they are more suited to classification tasks. The general structure of each created classifier is:

```
    Multi-Layer Perceptron Details:
Layer 1:  Linear Block (128, hidden_layer_size), Activation:  ReLU
Layer 2:  Linear Block (hidden_layer_size, 10), Activation:  Softmax
Learning Rate:  0.001 | 0.01 | 0.000001
Momentum:  0.0
Loss Function:  Cross-Entropy
Weight Initialization Type:  xavier
```

Classifier 1: hidden layer size = 64

Figure 30 displays the results of this model. Accuracy on test data: 0.8503

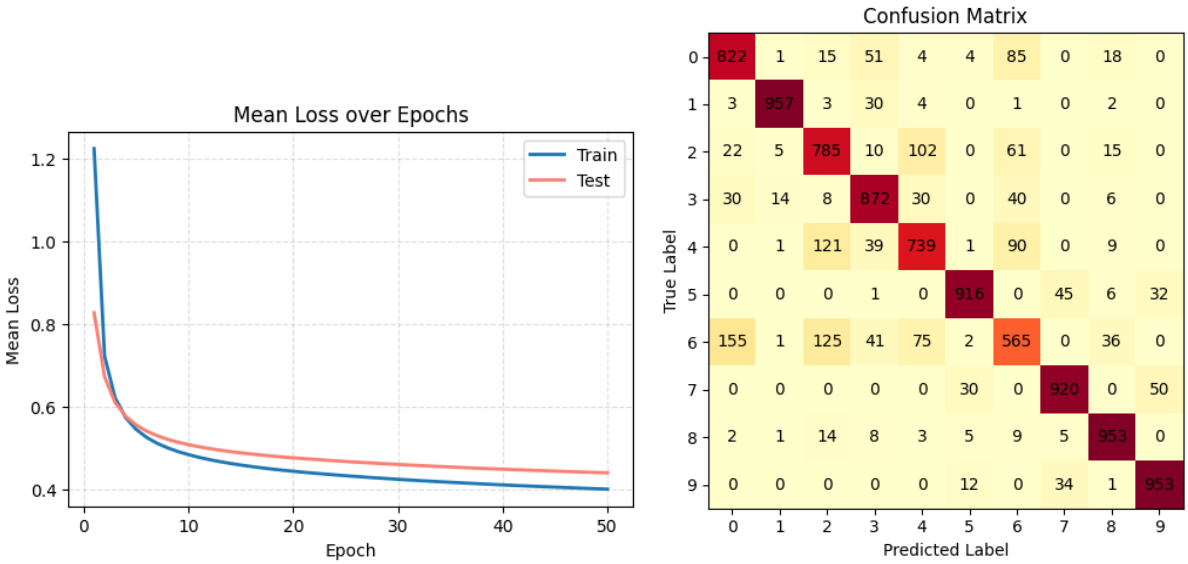


(**a**) Train and test Cross-Entropy over epochs (**b**) Confusion Matrix of the test data

Figure 30: Classifier model with hidden layer size = 64

Classifier 2: hidden layer size = 32

Figure 31 displays the results of this model. Accuracy on test data: 0.8482



(**a**) Train and test Cross-Entropy over epochs (**b**) Confusion Matrix of the test data

Figure 31: Classifier model with hidden layer size = 32

3.2.b Comparison with Official Results

The best results that I was able to find for this classification task (while using an MLP) was:
Accuracy = 0.870
Model: a single hidden layer of 100 neurons with tanh activation

To improve the MLP model on Fashion-MNIST, adjusting the architecture can allow the network to capture more complex patterns. Applying regularization techniques such as dropout or weight decay can reduce overfitting. Finally, increasing the training epochs can also help the model converge to a better solution.

References

- https://www.geeksforgeeks.org/deep-learning/mini-batch-gradient-descent-in-deep-learning/
- https://www.datacamp.com/tutorial/coefficient-of-determination
- http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/